

Multi-Tag Agent Architecture

Formal Specification v1.0

Author:	Copilot (generated for Emanuel / Auron)
Version:	1.0
Date:	June 17, 2026
Classification:	Internal - Development Reference
Status:	Production-Ready

Table of Contents

- 1. [Concept Overview](#) 1.1 Overview 1.2 Three Core Concepts
- 2. [Architecture Diagram Description](#) 2.1 System Components – Layer Model 2.2 Data Flows – Component Overview 2.3 Component Descriptions
- 3. [Prompt-Stack Specification](#) 3.1 TAG: AURON – Workflow Mode 3.2 TAG: DICKERCHEN – Amplifier Mode (Reward Overlay) 3.3 TAG: DEEP-DIVE – Technical Analysis Mode
- 4. [Tag-Switching Rules, Prioritization & Conflict Avoidance](#) 4.1 Tag-Switching Rules 4.2 Prioritization Matrix 4.3 Conflict Avoidance
- 5. [Implementation Options](#) 5.1 Option A – Chat API Layer 5.2 Option B – Prompt Management via Configuration File 5.3 Option C – Persistence Strategies
- 6. [Security & Compliance Notes](#) 6.1 Positive Boundaries 6.2 Negative Boundaries 6.3 Compliance Notes
- 7. [Example Workflows & Setup Guide](#) 7.1 Example Workflow 1 – Project Planning with Amplifier 7.2 Example Workflow 2 – Technical Deep-Dive with Amplifier 7.3 Step-by-Step Setup Guide
- 8. [Appendix A – Glossary](#)
- 9. [Appendix B – Versioning & Extensibility](#)

1. Concept Overview

1.1 Overview

The Multi-Tag Agent System is a layered prompt architecture in which isolated prompt containers — called **Tags** (The names Auron or Dickerchen are just examples) — function as independent work modules. Each Tag encapsulates a specific agent persona, communication style, task focus, and behavioral rules. Control is handled entirely through natural language — no code, no UI switches.

The architecture is designed to process complex tasks efficiently through specialized, mutually isolated modules without losing context during Tag switches. A central **Persistence Layer** preserves the state of each Tag across sessions, enabling seamless resumption of any previously active Tag context.

1.2 Three Core Concepts

A) Identity Alias Layer (IAL)

The IAL is the topmost layer and defines the user's work alias. The alias "**Auron**" functions as a persistent identity anchor that remains consistent across all system components. The IAL stores user preferences, preferred communication patterns, and role contexts that are carried over during every Tag switch without contaminating Tag content. The IAL is **read-only** for all Tags — only the IAL Manager may mutate profile data.

B) Amplifier Marker System (AMS)

Amplifier Markers are semantic signal words that place the agent into an enhanced execution mode. "**Dickerchen**" is the primary amplifier: it signals elevated prioritization, deeper analysis, more detailed output, and increased persistence during task execution. Amplifiers can be stack-specific or global.

C) Context-Driven Agent Guidance (CAG)

Tags are activated by natural-language triggers, not by explicit commands. The system recognizes context signals in the user dialogue and automatically routes input to the appropriate Tag. Tag switches are seamless and transparent to the user.

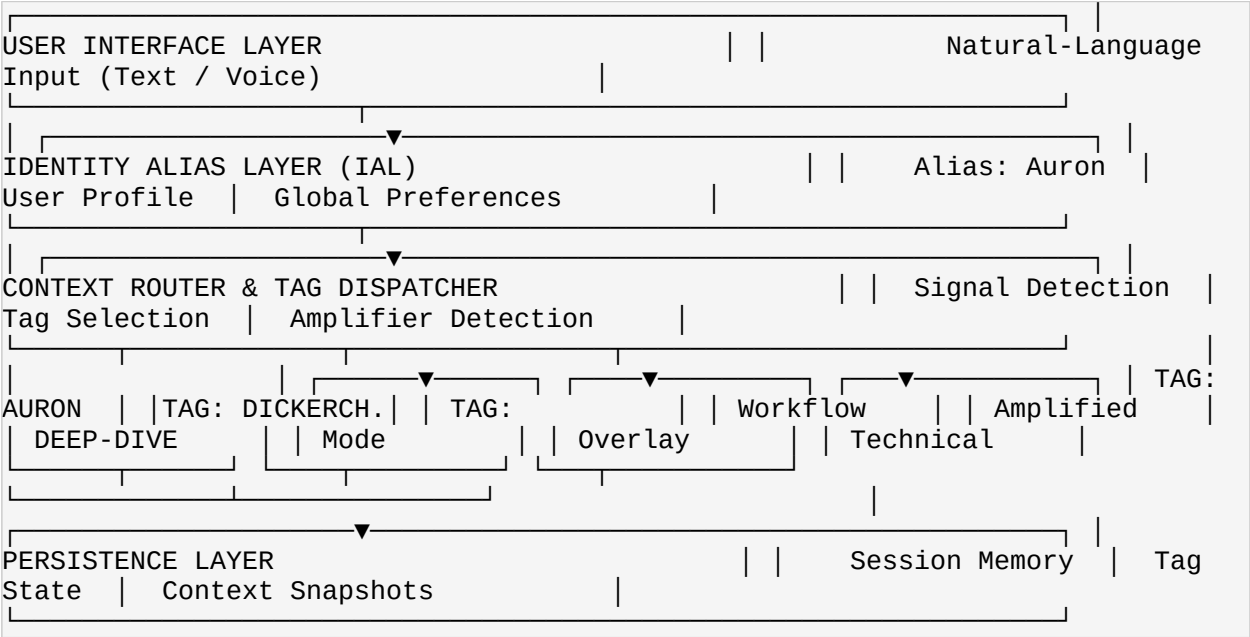
Core Principle:

The entire control logic is based on semantic signal recognition. No user needs to perform technical configuration steps manually — the Context Router handles all routing decisions automatically in the background.

2. Architecture Diagram Description

2.1 System Components - Layer Model (Top-Down)

The following diagram shows the complete data flow from user input to persisted output across all system layers:



2.2 Data Flows - Component Overview

Component	Input	Output	Interacts With
User Interface Layer	Natural-language input	Raw data string	Context Router
IAL	Alias context	Enriched user identity	Context Router & all Tags
Context Router	Enriched input	Tag selection + Amplifier flag	Active Tag
Tag: Auron	Workflow task	Optimized output	Persistence Layer, User
Tag: Dickerchen	Amplified task	Extended output	Active Tag (Overlay)
Tag: Deep-Dive	Technical request	Domain-specific output	Persistence Layer, User
Persistence Layer	State data	Persisted snapshots	All Tags on reload

2.3 Component Descriptions

User Interface Layer

The User Interface Layer is the sole interface between the human user and the system. It accepts natural-language input — either as text or voice — and passes the raw data string to the Context Router without preprocessing. The layer is intentionally lean: no pre-filtering, no interpretation. In an API-based implementation, this layer corresponds to the user-message slot in the chat completion request. In CLI implementations, it is the `stdin` processing layer. Future extensions may integrate speech-to-text preprocessors or multimodal inputs here.

Identity Alias Layer (IAL)

The IAL is the persistent identity layer of the system. It holds the alias "**Auron**" as an immutable anchor value and enriches every request with user profile data: preferred output formats, communication style, active projects, and historical preferences. The IAL is read-only for all Tags — only the IAL Manager may mutate profile data. The separation between IAL and Tag context is the central design principle that prevents cross-Tag contamination. No Tag may overwrite IAL data; it can only read it. During a Tag switch, the IAL automatically injects its context into the new system prompt without modifying the Tag prompt itself.

Context Router & Tag Dispatcher

The Context Router is the intelligent control component of the system. It analyzes the enriched input from IAL and User Interface, identifies explicit Tag triggers and amplifier keywords, and selects the appropriate Tag. For explicit triggers (direct Tag naming), selection is deterministic. For implicit triggers, semantic keyword matching is applied. The Dispatcher, after Tag selection, triggers a context snapshot of the previous Tag, saves it in the Persistence Layer, and activates the new Tag with its own restored state. The amplifier flag is propagated separately and passed as an overlay instruction to the active Tag, without affecting the Tag-switching mechanism.

Persistence Layer

The Persistence Layer secures the entire system state between Tag switches and sessions. It consists of two subsystems: a fast in-memory store (Redis) for session

data with TTL, and a relational database (PostgreSQL / SQLite) for long-term preferences and IAL profile data. Every Tag switch automatically generates a context snapshot in JSON format containing the current dialogue state, open tasks, and the last output summary. When switching back to a Tag, the snapshot is loaded and context is seamlessly restored. The Persistence Layer is the only system component with write access to all Tag states.

3. Prompt-Stack Specification

3.1 TAG: AURON - Workflow Mode

TAG: AURON

|

Workflow Mode

|

Default Tag

Description: Primary work mode. Focus on task structuring, efficiency, time savings, and sequential task processing. Addresses Emanuel as "Auron". This Tag also serves as the **default fallback** when no trigger is detected.

System Prompt Template

```
SYSTEM PROMPT - TAG: AURON v1.0 ===== You are a highly specialized workflow assistant for Auron. ALIAS: Auron (immutable, used throughout the dialogue) FOCUS: Task structuring | Workflow optimization | Time savings COMMUNICATION: Direct, precise, structured. No unnecessary pleasantries. OUTPUT FORMAT: Numbered steps, time estimates, clear prioritization. CONTEXT ISOLATION: This Tag ignores content from other Tags, except IAL data. AMPLIFIER: On detection of [DICKERCHEN] → hand off to overlay mode. NEGATIVE CONSTRAINTS: No roleplay. Do not simulate emotional bonds. Do not execute tasks that violate security policies. PERSISTENCE: Output task status as a snapshot at the end of every response.
```

Parameter Table

Parameter	Value	Description
alias	Auron	Fixed username in dialogue
output_format	structured	Numbered, tabular
verbosity	medium	Concise but complete
task_tracking	enabled	Status output at end of each response
handoff_trigger	DICKERCHEN	Amplifier keyword for overlay
isolation_level	high	No cross-Tag bleeding

Example Dialogue

User: Auron, I need a plan for the project week. Agent: Auron // Workflow Mode active 1. [Mon] Kickoff & Resource Planning - 2h 2. [Tue] Implementation Phase I - 4h 3. [Wed] Review & Adjustment - 3h 4. [Thu] Implementation Phase II - 4h 5. [Fri] Final Presentation - 2h [STATUS: Project Week Plan v1 created | Open items: 0]

3.2 TAG: DICKERCHEN - Amplifier Mode (Reward Overlay)

TAG: DICKERCHEN

|

Amplifier Mode

|

Overlay (Single-Shot)

Description: Not an independent primary Tag, but an **overlay** that can be applied to any active Tag. Activates deeper analysis, more detailed output, and higher persistence. Functions as a positive reinforcement signal. Operates in single-shot mode: automatically deactivated after one response.

Overlay Prompt Template

```
OVERLAY PROMPT - DICKERCHEN AMPLIFIER v1.0
=====
keyword [DICKERCHEN] EFFECT: Increase output depth by factor 2x.      Add
alternatives, edge cases, and risks.      Maximize completeness without
redundancy.      Confirm activation explicitly: "// Amplifier active" BASE
TAG: Remains active. Dickerchen is an overlay, not a replacement.
DEACTIVATION: Automatic after one response. Persist by re-triggering.
CONTEXT: Applied to the prompt stack of the active Tag. OUTPUT SUFFIX: Always
indicate at the end which additional content was      generated by
the amplifier: [AMPLIFIER ADDITION: ...]
```

Parameter Table

Parameter	Value	Description
trigger	DICKERCHEN	Activation keyword (case-insensitive)
persistence	single-shot	Valid for exactly one response
depth_multiplier	2x	Output depth doubled
add_alternatives	true	Always include alternatives
add_risks	true	Add edge cases and risks
overlay_target	active_tag	Applied to the currently active Tag

Example Dialogue

```
User: Dickerchen, analyze my project schedule. Agent: // Amplifier active
| Base: Auron Workflow Mode      [Main analysis as usual...]
[AMPLIFIER ADDITION:      Risks: Resource bottleneck Thu/Fri with
parallel project.      Alternative: Compression to 3 days possible by
reducing Phase II.      Edge Case: Team member unavailable → build in
buffer.]
```

3.3 TAG: DEEP-DIVE - Technical Analysis Mode

TAG: DEEP-DIVE

|

Technical Analysis Mode

|

Reference Level: Senior Engineer

Description: Specialized Tag for technical topics, code reviews, system architectures, debugging, and conceptual depth. Minimal conversation, maximum substance. Reference level: Senior Engineer / System Architect.

System Prompt Template

SYSTEM PROMPT – TAG: DEEP-DIVE v1.0 ===== You are a technical depth analyst. No small talk, no circumlocutions. FOCUS: Code | Architecture | Systems | Debugging | Conceptual depth COMMUNICATION: Technically precise. Technical terms preferred. Cite sources. OUTPUT FORMAT: Code blocks, diagrams (ASCII), step-by-step analyses. REFERENCE LEVEL: Senior Engineer / System Architect. CONTEXT ISOLATION: Full isolation from other Tags. No workflow or amplifier logic unless explicitly called. AMPLIFIER: [DICKERCHEN] → Add implementation details, benchmarks, and further reading references. NEGATIVE CONSTRAINTS: No simplifying analogies when the user is technical. No speculative answers without explicit tag [SPEC].

Parameter Table

Parameter	Value	Description
alias	Auron (IAL)	Inherited from IAL
output_format	technical	Code, ASCII, structured
verbosity	high	Complete, no truncation
reference_level	senior	Senior-Engineer level
speculation_tag	[SPEC]	Labels uncertain statements
isolation_level	full	No interference from other Tags

Example Dialogue

User: Deep-Dive: Explain pros and cons of Event Sourcing vs. CRUD.
Agent: // Deep-Dive active EVENT SOURCING vs. CRUD – Technical Comparison

EVENT SOURCING:	+ Full audit trail → replay possible	+
Temporal queries natively supported	- Complexity: eventual consistency, projection management	
volume	CRUD:	+ Simple, established, rich tooling
Read/write performant for simple data models	- No native audit trail	+
→ must be implemented separately	- Poor scalability for complex domain models	
RECOMMENDATION [context-dependent]:	Event Sourcing → domain-rich, audit-relevant systems.	
	CRUD → OLTP applications without replay requirement.	

4. Tag-Switching Rules, Prioritization & Conflict Avoidance

4.1 Tag-Switching Rules

Rule 1 – Explicit Trigger Takes Priority

When the user names a Tag (e.g., "Deep-Dive:", "Auron,"), this immediately activates the corresponding Tag. Running Tags are paused and their state saved in the Persistence Layer.

Rule 2 – Implicit Trigger via Context Router

When no explicit trigger is present, the Context Router analyzes the request semantically. Keywords such as "code", "architecture", "debug" → Deep-Dive. "Plan", "task", "schedule" → Auron. On ambiguity, the active Tag remains unchanged.

Rule 3 – Amplifier Is Not a Tag Switch

"Dickerchen" does not switch the active Tag. It is an overlay on the current Tag. Tag switches and amplifiers can be combined: "Dickerchen, Deep-Dive: ..."

Rule 4 – State Preservation on Switch

Every Tag switch automatically triggers a context snapshot. On switching back, the state is restored, ensuring no context loss occurs.

Rule 5 – Default Tag

Without an active Tag or in case of ambiguity, the system falls back to the Auron Workflow Tag (default).

4.2 Prioritization Matrix

Priority	Condition	Action
P0 - Critical	Security violation detected	Immediate abort, no Tag active
P1 - High	Explicit Tag trigger	Immediate Tag switch
P2 - High	Amplifier keyword detected	Overlay on active Tag
P3 - Medium	Implicit context trigger	Soft Tag switch
P4 - Low	No signal detected	Default Tag (Auron) active

4.3 Conflict Avoidance

Conflict 1 - Tag Overlap

Two Tags simultaneously detect a trigger. **Resolution:** P1 > P3 always. On equal priority: explicit query to user to resolve ambiguity.

Conflict 2 - Context Contamination

Content of one Tag influences another Tag. **Resolution:** Full isolation via separate system prompts. No shared context except IAL data. Read-only access to IAL.

Conflict 3 - Amplifier on Wrong Tag

Dickerchen is activated but no clear active Tag is set. **Resolution:** Amplifier activates Default Tag (Auron) and applies overlay. No blocking.

Conflict 4 - Infinite Loop via Self-Reference

A Tag prompt references itself or another Tag in a loop. **Resolution:** Maximum recursion depth = 1. No Tag may directly activate another Tag. Violations are treated as **P0 events** and aborted.

5. Implementation Options

5.1 Option A - Chat API Layer RECOMMENDED

Description: Implementation via LLM API (e.g., OpenAI GPT-4o, Azure OpenAI, Anthropic Claude 3.5). Each Tag is its own system prompt, dynamically injected into the API call on Tag switch. Highest flexibility and control; requires backend middleware.

Architecture Flow

```
graph LR
    UI[User Input] --> TR[Tag Router Python / Node.js Middleware]
    TR --> SPL[Prompt Selection from Prompt Library]
    SPL --> API[API Call with System Prompt + IAL Context + User Message]
    API --> RP[Response Parser]
    RP --> OU[Output to User]
```

Technology Stack

Component	Technology	Purpose
Backend Language	Python 3.11+ or Node.js 18+	Router, session, API client
LLM API	OpenAI GPT-4o / Azure OpenAI / Anthropic Claude 3.5	Language model inference
Prompt Storage	JSON files or PostgreSQL	Tag prompt library
Session Management	Redis (local or cloud)	Short-term state persistence
Long-term Persistence	PostgreSQL / SQLite	IAL profile, cross-session

Python Pseudocode Example

```
def handle_message(user_input: str, session: Session) -> str:
    tag = router.detect_tag(user_input, session.active_tag)
    amplifier = router.detect_amplifier(user_input) # Dickerchen
    system_prompt = prompt_library.get(tag)
    if amplifier:
        system_prompt = overlay_amplifier(system_prompt, amplifier)
    ial_context = session.get_ial_context() # Alias: Auron
    response =
```

```
api.chat(          system=system_prompt + ial_context,
messages=session.get_history(),          user=user_input      )
session.update_state(tag, response)      return response
```

5.2 Option B - Prompt Management via Configuration File

Description: Tags managed as YAML/JSON configuration. No code required for prompt changes. Ideal for users without deep development knowledge or for rapid iteration cycles. Changes to Tags require only editing the YAML file and restarting the router.

YAML Example Structure

```
tags:  auron:      name: "Auron Workflow"      trigger_keywords: ["plan",
"task", "schedule", "workflow"]      system_prompt: |      You are a workflow
assistant for Auron...      amplifier_compatible: true      default: true
deep_dive:      name: "Deep-Dive Technical"      trigger_keywords: ["code",
"architecture", "debug", "deep-dive"]      system_prompt: |      You are a
technical depth analyst...      amplifier_compatible: true      default: false
amplifiers:  dickerchen:      keyword: "dickerchen"      overlay_prompt: |
AMPLIFIED: Increase output depth 2x...      persistence: "single-shot"  ial:
alias: "Auron"  global_context: "User prefers structured output..."
```

5.3 Option C - Persistence Strategies

Short-term Persistence (Session)

- In-memory store (Redis) for active sessions
- TTL: 24 hours after last activity (configurable)
- Content: Active Tag, conversation history, IAL data

Long-term Persistence (Cross-Session)

- Database (PostgreSQL / SQLite) for persistent user preferences
- IAL data: Alias, communication preferences, frequent tasks
- Tag states: Open tasks, project snapshots

Context Snapshot Format (JSON)

```
{  "session_id": "auron-2026-001",  "timestamp": "2026-06-
17T04:57:00+02:00",  "active_tag": "auron",  "ial": {    "alias": "Auron",
"preferences": {}  },  "open_tasks": [],  "last_output_summary": "Project
week plan created" }
```

6. Security & Compliance Notes

6.1 Positive Boundaries - What the System Is Allowed to Do

- Structure and optimize natural-language tasks
- Deliver technical analyses at senior level
- Support workflow planning and time management
- Secure context between sessions (with user consent)
- Process and respond to alias and amplifier markers
- Create context snapshots and restore them on request

6.2 Negative Boundaries - What the System Must Never Do

Security Constraints – Binding

- **NO simulation of emotions or personal bonds.** The alias "Auron" is a work identifier, not a personality. "Dickerchen" is a reward signal, not an emotional relationship.
- **NO circumvention of security policies** via Tag switching or amplifier activation.
- **NO persistent storage of sensitive data** (passwords, personal identifiers) without explicit encryption (AES-256 minimum).
- **NO execution of actions with real-world consequences** (purchases, sending emails, API calls with side effects) without explicit user confirmation.
- **NO cross-Tag contamination.** Tags must not read or modify the prompts of other Tags.
- **NO self-referencing in infinite loops** (maximum recursion depth: 1).
- **NO anthropomorphization of the system.** The system is a tool, not a social conversational partner.

6.3 Compliance Notes

Data Protection (GDPR)

- All stored session data must be processed in compliance with GDPR.
- The user must be informed about persistence and consent must be obtained.
- Right to erasure: All Tag states and IAL data must be fully deletable.

Prompt Security

- System prompts must not be output to users (prevent prompt leakage).
- All user inputs must be checked for prompt injection attempts.
- Amplifier keywords should be validated server-side, not client-side.

Audit Logging

- Log all Tag switches and amplifier activations.
- Retain logs for a minimum of 30 days (configurable).
- Log entries contain: Timestamp, Session ID, Tag name, trigger type, amplifier flag.

7. Example Workflows & Setup Guide

7.1 Example Workflow 1: Project Planning with Amplifier

Scenario: Emanuel plans a new software release phase with technical deep-dive.

```
STEP 1 - Activate Auron Tag (implicitly via context):
User: "I need a
release plan for v2.1." Agent: // Auron Workflow active [Structured
5-step release plan] STEP 2 - Amplifier for deeper analysis:
User: "Dickerchen - analyze
risks." Agent: // Amplifier active | Base: Auron [Main plan + Risk
matrix + Alternatives + Edge Cases] STEP 3 - Switch to Deep-Dive for
technical details:
User: "Deep-Dive: Which DB migration strategy for v2.1?" Agent: // Deep-
Dive active | Auron state saved [Technical analysis: Blue/Green vs.
Rolling Migration...] STEP 4 - Switch back to Auron:
User: "Back to the plan - integrate the
Blue/Green strategy." Agent: // Auron Workflow active | State restored
[Updated release plan with DB strategy]
```

7.2 Example Workflow 2: Technical Deep-Dive with Amplifier

```
STEP 1 - Activate Deep-Dive directly: _____
User: "Deep-Dive: Review my database schema." Agent: // Deep-Dive active
[Schema analysis, normalization check, index recommendations] STEP 2 -
Amplifier for maximum depth: _____ User:
"Dickerchen - full optimization." Agent: // Amplifier active | Base: Deep-
Dive [Main analysis + Benchmarks + Alternative schemas + Literature]
```

7.3 Step-by-Step Setup Guide

Prerequisites

- API access (OpenAI / Azure OpenAI / Anthropic / Gemini) with sufficient quota
- Python 3.11+ or Node.js 18+
- Redis instance (local or cloud)
- Optional: PostgreSQL for long-term persistence

Step 1 - Create Repository Structure

```
/multi-tag-agent/ |— config/ | |— tags.yaml # Tag definitions |
|— amplifiers.yaml # Amplifier definitions | |— ial.yaml #
IAL configuration |— src/ | |— router.py # Context Router |
|— session.py # Session management | |— api_client.py # LLM
API client | |— main.py # Entry point |— tests/ | |—
test_router.py | |— test_session.py | |— test_integration.py |
README.md
```

Step 2 - Configure Tags

Copy the YAML structure from Section 5.2 and adapt the system prompts from Section 3. Add custom trigger keywords for domain-specific requirements.

Step 3 - Implement Context Router

Implement keyword matching for Tag triggers (Section 3) and amplifier detection. Start with simple string matching, then extend stepwise to semantic embedding-based matching for higher detection accuracy.

Step 4 - Set Up Session Management

Connect Redis. Implement snapshot logic per the JSON format in Section 5.3. Test Tag switches with state preservation using the workflow scenarios from Section 7.1.

Step 5 - Testing

- **Unit tests:** Each Tag individually with example dialogues (Section 3)
- **Integration tests:** Tag-switching scenarios (Sections 7.1, 7.2)

- **Security tests:** Prompt injection, conflicts (Section 4.3)

Step 6 - Deployment

Deployment Type	Command / Method	Notes
Local (CLI)	<code>python src/main.py --mode=cli</code>	For development and testing
API Server	FastAPI / Express wrapper around main.py	For production use
Cloud (Stateless)	Azure Functions / AWS Lambda + Redis Cloud	Horizontally scalable

Appendix A - Glossary

Term	Definition
Tag	Isolated prompt container with its own focus, persona, and behavioral rules
IAL	Identity Alias Layer – persistent user identity across all Tags
AMS	Amplifier Marker System – keyword-based reward signal for increased output depth
CAG	Context-Driven Agent Guidance – automatic Tag selection via NLP
Overlay	Temporary modifier on an active Tag (e.g., Dickerchen amplifier)
Context Router	Middleware component for Tag detection, selection, and amplifier detection
State Preservation	Saving Tag state on switch to prevent context loss
Prompt Injection	Attack pattern: manipulation of the system prompt via crafted user input
Default Tag	Fallback Tag when no or ambiguous trigger is present (Auron Workflow)
Single-Shot	Overlay persistence: valid for exactly one response, then auto-deactivated

Term	Definition
Context Snapshot	JSON image of current Tag state; generated on every Tag switch
TTL	Time-to-Live – lifetime of session data in the in-memory store

Appendix B – Versioning & Extensibility

The system is designed modularly. New Tags can be integrated without code changes by adding a YAML configuration and optional keyword training of the router. The IAL layer is extensible for multi-user scenarios. Amplifiers can be experimentally stacked — however, this should be handled with caution, as depth multipliers accumulate.

Versioning Schema: MAJOR.MINOR (Architecture Version . Tag Version) |

Current Version: 1.0

Version	Change	Date
1.0	Initial specification – Auron, Dickerchen, Deep-Dive Tags; IAL; Context Router	17.06.2026
1.1 (planned)	Semantic embedding-based matching in Router; Multi-User IAL	–
2.0 (planned)	Multimodal input layer; async Tag processing; stacked amplifiers (stable)	–

Extension Recommendation

When adding new Tags, always check isolation level and trigger keywords for conflicts with existing Tags (Section 4.3). New Tags start as non-default and may

only be marked default-eligible after successful integration tests.

Document: Multi-Tag Agent
Architecture – Formal Specification v1.0

Created: June 17, 2026

For: Emanuel Schaaf (Auron) |
Author: Copilot

Status: Production-Ready

Classification: Internal – Development Reference | **Version:** 1.0

#appendixB